

Week 2 - Day 9b Supplementary: Debugging and Comprehensions

Exercise 1: Understanding Errors []

Oops! Each snippet of the following code crashes. Run the code, **scroll to the bottom** of the error message, and read it. See if you can find the line of code where the error occurred by reading the messages!

Try to fix the errors! **DO NOT** copy your code from the previous labs, but make **minor** changes to the code here.

1.1 The function `print_list` is supposed to print out all the elements in a nested list but something is wrong. Fix the function.

```
# TODO: Fix the function
def print_list(lst):
    if type(lst) == list:
        for i in lst:
            print_list(lst[i])
    else:
        print(lst)
```

1.2 The function `removeLetter` is supposed to remove all occurrences of a letter in a string but something is wrong. Fix the function.

```
# TODO: Fix the function
def remove_letter(s, letter):
    if s[0] == letter:
        return remove_letter(s[1:], letter)
    return s[0] + remove_letter(s[1:], letter)
```

1.3 The function `sum_list` is supposed to sum up all elements in a nested list but something is wrong. Fix the function.

```
# TODO: Fix the function
def sum_list(lst):
    if type(lst) == list:
        summ = 0
        for i in lst:
            summ += sum_list(i)
    else:
        return lst
```

Exercise 2: List + Dictionary Comprehensions []

2.1 Write a code that counts the occurrences of each numbers in a list of integers. Use loops.

```
# TODO: Your code here
numbers = [1, 2, 2, 2, 3, 3, 3, 4, 4, 4, 4, 4]
```

2.1 Write a code that counts the occurrences of each numbers in a list of integers. Use dictionary comprehension.

```
# TODO: Your code here
numbers = [1, 2, 2, 2, 3, 3, 3, 4, 4, 4, 4, 4]
```

2.3 Write code that extracts the first character of each word in a sentence. Use list comprehension.

```
# TODO: Your code here
sentence = "AddisCoder is helping Ethiopian high schoolers to achieve their dreams."
```

2.4 Write code that takes list of strings and creates a new list with the length of each string using list comprehension.

```
# TODO: Your code here
names = ["Alemu", "Luel", "Yoseph", "Adoni", "Meseret", "Tsion"]
```

2.5 Write code that finds all the common elements between two given lists. Use list comprehension.

```
# TODO: Your code here
list1 = [5, 2, 3, 4, 6, 8, 7, 9]
list2 = [4, 5, 6, 7, 8, 3, 1, 10]
```

Exercise 3: Anagrams []

An [anagram](#) is a word or phrase formed by rearranging the letters of a different word or phrase using all the original letters exactly once.

Look here for more info:

3.1 Implement a function `is_anagram` that checks if two words are anagrams of each other.

```
# TODO: Your code here
def is_anagram(word1, word2):
    pass

# Use this cell to test your solution
assert is_anagram("listen", "silent") == True
```

```

assert is_anagram("hello", "world") == False
assert is_anagram("debit card", "bad credit") == True
assert is_anagram("astronomer", "moon starrer") == True
assert is_anagram("hello", "world") == False

print("All tests passed. ☐")

```

3.2 Given a some word and a list of words, print all anagrams of the given word in the list. Use list comprehension.

```

# TODO: Your code here
given_word = "listen"
word_list = ["enlist", "silent", "hello", "inlets", "world", "tinsel"]

```

Exercise 4: Matrix Transpose [☐☐☐]

A matrix $m \times n$ matrix A is a rectangular array of numbers with m rows and n columns. Each element A_{ij} is the element in the i -th row and j -th column.

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

The **transpose** of a matrix A^T is a matrix that is obtained by flipping the matrix over its diagonal. This is equivalent to switching the row and column indices of the matrix, i.e. $A_{ij}^T = A_{ji}$. The full matrix transpose is defined as

$$A^T = \begin{bmatrix} a_{11} & a_{21} & \cdots & a_{m1} \\ a_{12} & a_{22} & \cdots & a_{m2} \\ \vdots & \vdots & \ddots & \vdots \\ a_{1n} & a_{2n} & \cdots & a_{nm} \end{bmatrix}$$

4.1 Implement a function **transpose** that takes a matrix and returns its transpose. Use loops.

```

# TODO: Your code here
def transpose(matrix):
    pass

# Use this code to test you solution
assert transpose([[]]) == [[]]
assert transpose([[1]]) == [[1]]
assert transpose([[1, 2], [3, 4], [5, 6]]) == [[1, 3, 5], [2, 4, 6]]
assert transpose([[1, 2, 3], [4, 5, 6]]) == [[1, 4], [2, 5], [3, 6]]
assert transpose([[1, 2, 3, 4], [4, 5, 6, 8]]) == [[1, 4], [2, 5], [3, 6], [4, 8]]

```

```
print("All tests passed. ☐")
```

4.1 Re-implement the function `transpose` that takes a matrix and returns its transpose but use list comprehension.

```
# TODO: Your code here
def transpose(matrix):
    pass

# Use this code to test you solution
assert transpose([[]]) == [[]]
assert transpose([[1]]) == [[1]]
assert transpose([[1, 2], [3, 4], [5, 6]]) == [[1, 3, 5], [2, 4, 6]]
assert transpose([[1, 2, 3], [4, 5, 6]]) == [[1, 4], [2, 5], [3, 6]]
assert transpose([[1, 2, 3, 4], [4, 5, 6, 8]]) == [[1, 4], [2, 5], [3, 6], [4, 8]]

print("All tests passed. ☐")
```

Congratulations ☐